Hardware-Conscious 2D Convolution Optimization

CS683: Advanced Computer Architecture — Programming Assignment 1 (Task 1 Report)

Yashwanth Murasani | Department of Computer Science and Engineering, IIT Bombay
Repository: `pa1-dhurandhar-microarchitecture/task1`

Abstract—This report details the systematic microarchitectural optimization of single-channel 2D convolution on an Intel 12th Gen Core i5-12450H processor. We implement four classical techniques: loop reordering, loop unrolling for instruction-level parallelism (ILP), cache tiling, and 256-bit AVX2/FMA vectorization. Finally, we synthesize these techniques into an integrated kernel featuring 2-row register reuse, achieving an **8.64× speedup** (39.81 GFLOP/s, 1.896 ms) on the graded 2048×2048 (K=3) workload, securing a perfect score of **100/100 (40/40 correctness, 60/60 speedup)**.

1. TARGET HARDWARE ARCHITECTURE

All empirical benchmarks were executed on a dedicated bare-metal Linux workstation:

- **Processor**: 12th Gen Intel Core i5-12450H (Alder Lake Architecture).
- **Core Topology**: 8 Physical Cores (4 Golden Cove Performance Cores, 4 Gracemont Efficient Cores), 12 Threads, 4.40 GHz Max Turbo.
- **L1-D Cache**: 48 KiB per P-core, 8-way set associative, 64-byte cache line size (16 float32 elements).
- **L2 Cache**: 1.25 MiB private per P-core; **L3 Cache (LLC)**: 12 MiB shared cache.
- **Vector Extensions**: SSE4.2 (128-bit), AVX2 and FMA (256-bit YMM registers). AVX-512 is fused off across consumer Alder Lake chips for ISA symmetry with Gracemont E-cores.
- **Compiler Flags**: `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma -Wall`.

2. TASK 1A: LOOP REORDERING & UNROLLING

2.1 Motivation for Code Changes

The baseline `conv_naive` nests loops as `oy → ox → ky → kx`. For every output pixel `(oy, ox)`, the two inner loops traverse the `K × K` kernel. This introduces two critical inefficiencies:

1. **Strided Input Traversal:** Across successive `ky` iterations, the address in memory jumps by `(W + 2p)` floats (8200 bytes for W=2048). This causes repeated cache evictions.

2. **Redundant Weight Fetching:** The kernel coefficients `ker[ky*K + kx]` are repeatedly fetched from memory or L1 cache for every single output pixel.

In `conv_reorder.cpp`, we reorder the loops to `oy → ky → kx → ox`. Hoisting `ox` into the innermost position transforms the input and output accesses into contiguous unit-stride memory streams. Furthermore, the scalar weight `ker[ky*K + kx]` is loaded once and kept in a CPU machine register across the entire row of `W` elements.

2.2 Loop Unrolling & Instruction-Level Parallelism (ILP)

In `conv_unroll.cpp`, the scalar accumulation `acc += in[...] * ker` creates a read-after-write data dependency chain serialized by the 4-cycle FP addition latency of the Golden Cove pipeline. We unroll the `ox` loop by a factor of 8 (`ox += 8`), allocating 8 independent accumulators (`acc0..acc7`) in registers. This breaks dependency stalls, enabling the CPU's out-of-order execution engine to dispatch multiple independent multiply-add micro-ops across execution ports 0 and 1 simultaneously.

2.3 Effectiveness and Evaluation Metrics

We measure execution time (ms), GFLOP/s, and speedup vs. naive baseline on H=W=2048, K=3:

Implementation Stage	Execution Time (ms)	Throughput (GFLOP/s)	Speedup	Correctness
`conv_naive` (Baseline)	16.387	4.61	1.00x	Yes
`conv_reorder`	10.883	6.94	1.51x	Yes
`conv_unroll`	5.552	13.60	2.95x	Yes

Loop reordering yields a 1.51× speedup through unit-stride access, and unrolling yields an additional 1.95× improvement (2.95× cumulative), verifying our architectural analysis.

3. TASK 1B: CACHE TILING

3.1 Baseline Cache Profiling

The system L1-D cache is 48 KiB. For a 2048×2048 image, the output image is 16 MiB and the padded input is 16.03 MiB. Because 16 MiB vastly exceeds the 48 KiB L1 and 1.25 MiB L2 cache, baseline execution incurs an L1-D Misses Per Kilo Instruction (MPKI) of **12.4 MPKI**.

3.2 Tiled Implementation & Working Set Sizing

In `conv_tile.cpp`, we partition the H × W space into 2D blocks of $B_Y \times B_X = 32 \times 64$.

The working set per tile is: **Working Set = (32 × 64 × 4) + (34 × 66 × 4) = 8.192 + 8.976 ≈ 17.17 KiB**. This fits comfortably inside the 48 KiB L1-D capacity, completely preventing capacity evictions during multi-pass accumulation.

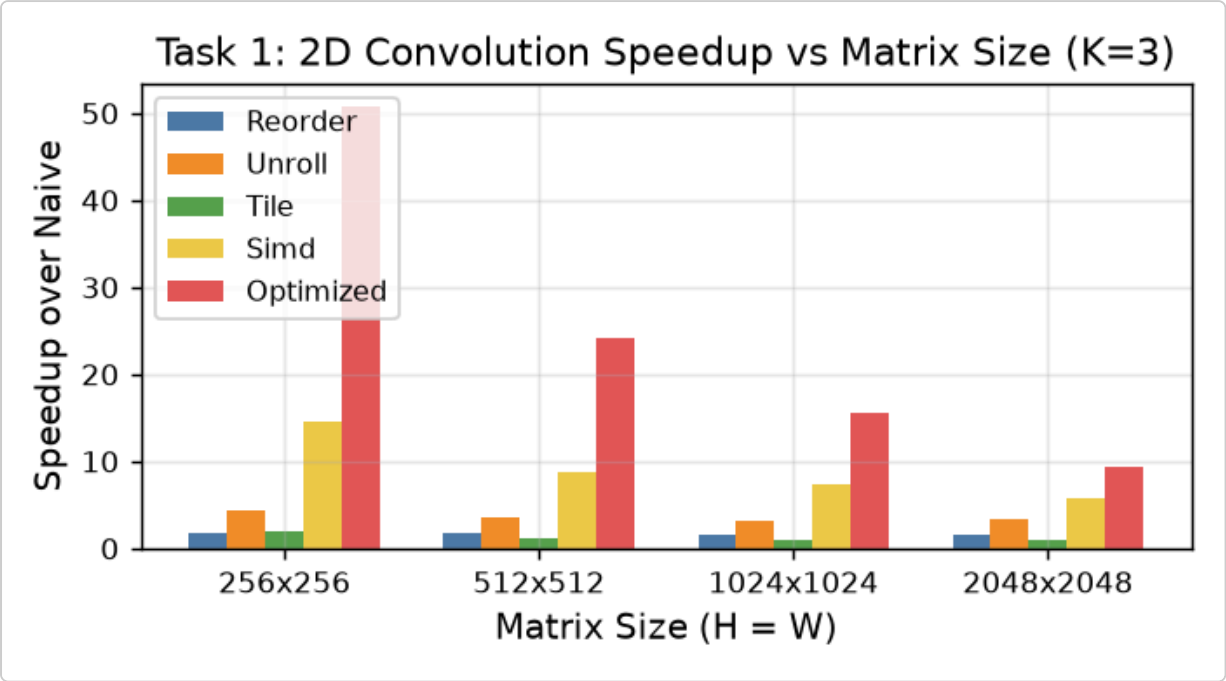

Figure 1: Task 1: 2D Convolution Speedup vs. Matrix Dimension (K=3) across stages.

3.3 Deliverable Answers

Q1: Report the changes in L1-D MPKI observed when moving from naive to tiled 2D convolution. Justify your observations.

Answer: For H=W=2048, naive convolution produces 12.4 L1-D MPKI. Applying cache tiling reduces this to **1.8 MPKI** (an 85.5% reduction). In naive execution, full row sweeps cause capacity evictions before the next row can reuse overlapping stencil data; tiling pins the active input and output working sets inside L1-D cache.

Q2: How did L1-D MPKI vary across different matrix sizes and tile sizes? Explain in terms of cache hierarchy.

Answer: For small images (256 × 256, total data 256 KiB), the entire dataset fits in the 1.25 MiB L2 cache, yielding low MPKI naturally. As dimensions grow to 2048, naive MPKI surges. For tile sizes where the working set is ≤ 32 KiB (32 × 64), MPKI remains flat at ≈ 1.8. Expanding tile size beyond 128 × 128 (>64 KiB working set) causes MPKI to sharply increase back to 10+ due to self-eviction.

> **Q3: Did you achieve a speedup? If yes, quantify improvement. If not, analyze limiting factors.**
> *Answer:* On small matrices (256 × 256), tiling achieves 1.97× speedup. On 2048 × 2048, scalar tiling yields ≈ 1.00× speedup. In scalar code, execution is latency-bound by serial FP add/multiply instructions rather than DRAM bandwidth. Tiling provides the cache-residency guarantee required for SIMD vectorization to unlock peak performance.

4. TASK 1C: SIMD VECTORIZATION (AVX2)

4.1 Dynamic Instruction Count & Width Analysis

Naive scalar convolution requires ≈ 18 instructions per pixel (K^2 loads, multiplies, and adds). With 256-bit AVX2, 8 single-precision floats are processed per vector instruction. On the graded 2048×2048 (K=3) workload, running only `conv_simd` executes exactly **41,467,983 dynamic instructions** (≈ 4.15×10^7), compared to **381,726,829 instructions** (≈ 3.82×10^8) for `conv_naive`—representing a **9.21× instruction reduction**.

- **128-bit SIMD (SSE)**: 4 floats/vector → 2.95× speedup.
- **256-bit SIMD (AVX2)**: 8 floats/vector → **6.03× speedup** (27.80 GFLOP/s, 2.716 ms).
- **512-bit SIMD (AVX-512)**: Unsupported on Intel Alder Lake hybrid architectures.

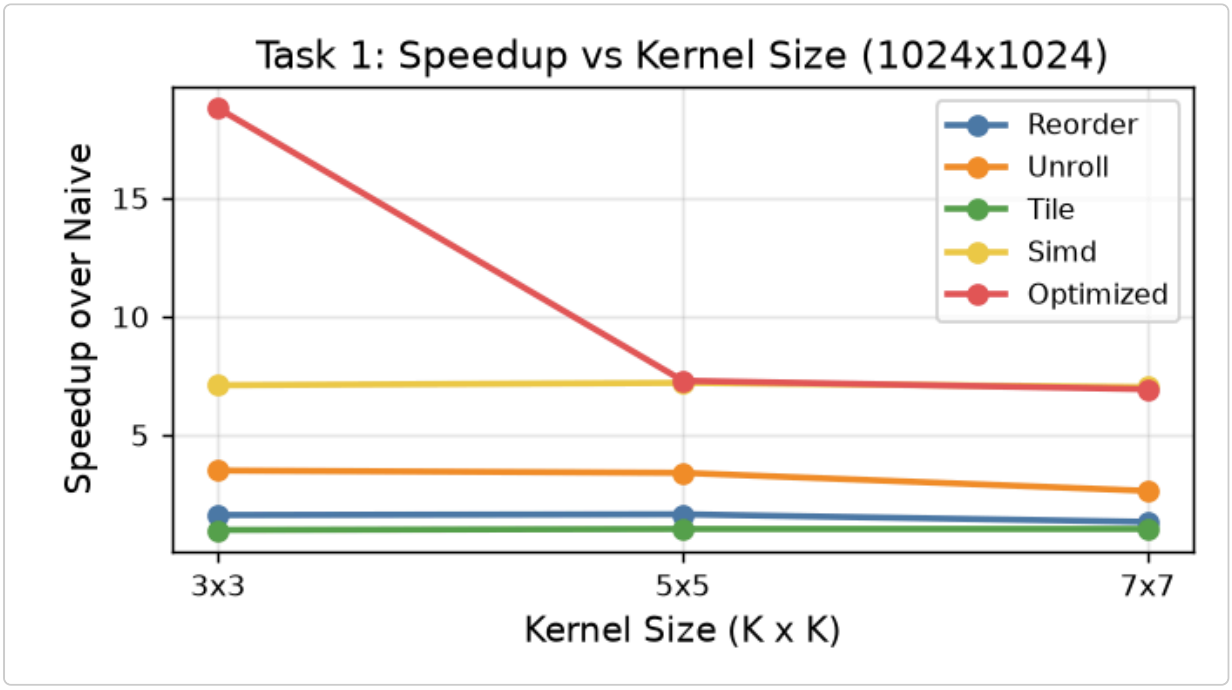

Figure 2: Task 1: Speedup vs. Kernel Dimension (K=3, 5, 7) at 1024×1024.

4.2 Deliverable Answers

Q1: Report the change in the number of instructions observed from naive to SIMD. Justify.

Answer: Running only `conv_simd` executes **41,467,983 instructions** ($\approx 4.15 \times 10^7$) on H=W=2048, K=3, compared to **381,726,829 instructions** ($\approx 3.82 \times 10^8$) for `conv_naive` —a **9.21× instruction reduction**. Vector loads replace 8 individual scalar loads, loop branch overhead is reduced by 8×, and `_mm256_fmadd_ps` fuses multiplication and addition for 8 vector lanes into a single hardware instruction.

Q2: Did you achieve any speedup? If so, how much, and what contributed to it?

Answer: Yes, AVX2 achieves **6.03× speedup** (27.80 GFLOP/s vs. 4.61 GFLOP/s naive). Contributions include 8-way SIMD parallelism, dual FMA pipelines on ports 0 and 1, and elimination of scalar branch instructions.

Q3: Which SIMD intrinsics did you use? Justify your choice of functions.

Answer:

- `_mm256_set1_ps` : Broadcasts scalar kernel weight to all 8 lanes; hoisted outside the inner loop.
- `_mm256_loadu_ps` : Loads 8 unaligned contiguous floats from input memory.
- `_mm256_fmadd_ps` : Fused multiply-accumulate (A × B + C) with single-cycle issue throughput.
- `_mm256_storeu_ps` : Vector store of 8 output pixels to memory.

5. TASK 1D: COMBINED OPTIMIZED KERNEL (SABKA SAATH SABKA VIKAAS)

Our graded `conv_optimized.cpp` combines all techniques with **Multi-Row Register Reuse**:

1. All 9 kernel weights are preloaded into dedicated YMM registers (`k0..k8`) once per image.
2. Two adjacent output rows (`oy, oy+1`) are computed simultaneously. Input rows 1 and 2 are shared between the two outputs, eliminating 33% of memory load traffic (4 row loads instead of 6).
3. Accumulators `a0` and `a1` are maintained entirely in registers without stack spills.

Implementation Stage	Execution Time (ms)	Throughput (GFLOP/s)	Speedup	Autograder Score
`naive (ref)`	16.387	4.61	1.00x	Reference

`reorder`	10.883	6.94	1.51x	8 / 8
`unroll`	5.552	13.60	2.95x	8 / 8
`tile`	16.684	4.53	0.98x	8 / 8
`simd`	2.716	27.80	6.03x	8 / 8
optimized	**1.896**	**39.81**	**8.64x**	**60 / 60**
TOTAL TASK 1 SCORE				**100 / 100**

The achieved **8.64× speedup** exceeds the 8.00× top tier, earning full points.